

Documentación de seguridad — Ecosistema BabyWolf

Materia: Desarrollo para Dispositivos Inteligentes · Evaluación 2 · Mayo–Agosto 2026

1. Validación de origen en BroadcastChannel

La PWA de Smart TV recibe la noticia seleccionada por dos caminos. **Ninguno de los dos llega a la pantalla sin pasar antes por dos filtros: de dónde viene y qué forma tiene.**

Implementado en `tv_pwa/js/sync.js`:

```
canal.onmessage = (event) => {  
  // BroadcastChannel es same-origin por diseño, pero la validación es  
  // explícita: si el origen no es el nuestro, el mensaje no se procesa.  
  if (event.origin && event.origin !== location.origin) {  
    console.warn('[seguridad] mensaje descartado, origen no confiable:', event.origin);  
    return;  
  }  
  
  const msg = validarMensaje(event.data);  
  if (!msg) {  
    console.warn('[seguridad] mensaje descartado, forma inválida');  
    return;  
  }  
  alRecibir(msg);  
};
```

Por qué se valida algo que el navegador ya garantiza

`BroadcastChannel` sólo entrega mensajes entre contextos del mismo origen, así que en teoría `event.origin` siempre coincide. La comprobación se deja escrita de todos modos por dos razones:

1. **No depender de una garantía implícita.** Si mañana el canal se sustituye por `postMessage` entre ventanas o por un `iframe`, la validación ya está en su sitio; sin ella, ese cambio abriría un agujero silencioso.
2. **La forma del mensaje sí es responsabilidad nuestra.** El origen correcto no implica contenido correcto: otra pestaña de la misma app, o código inyectado en ella, puede emitir cualquier cosa. Por eso `validarMensaje()` exige un `slug` string de 1 a 200 caracteres y una `category` de máximo 60, y descarta todo lo demás.

Defensa contra XSS en el renderizado

El contenido de las noticias viene de una API y se pinta **siempre con `textContent` , nunca con `innerHTML`** (`tv_pwa/js/app.js`). Aunque un título contuviera `<script>` , el navegador lo mostraría como texto.

Las portadas pasan por `urlSegura()` (`tv_pwa/js/api.js`), que sólo acepta `https:` . Así una URL `javascript:` o `data:` no puede acabar inyectada en la regla CSS de `background-image` .

2. LFPDPPP — datos personales tratados

Ley Federal de Protección de Datos Personales en Posesión de los Particulares.

Dato	¿Es personal?	Base legal	Dónde vive
Noticias del blog (título, contenido, portada, categoría)	No	Contenido editorial público	API pública
<code>author_id</code> de cada noticia	Sí, identificador	Art. 8 — consentimiento del autor al publicar	Base de datos del blog
Temas y número de noticias del wearable	No	Contenido editorial público	Se consultan a la API, se guardan en memoria
Noticias leídas y minutos de sesión	No en esta entrega	—	Sólo en memoria
Noticia seleccionada (<code>slug</code>)	No	—	Memoria del hub

Declaración honesta del alcance. El wearable muestra datos reales del blog —qué temas existen y cuántas noticias tiene cada uno—, que son contenido editorial público, no datos de ninguna persona. Lo único que simula es el acto de leer, y ese contador vive en memoria y muere al cerrar la app. No hay cuentas, ni login, ni sensores biométricos en ninguno de los tres dispositivos. Por eso el ecosistema **no trata datos personales de usuarios finales**.

Si el simulador se sustituyera por sensores reales (ritmo cardíaco, pasos), esos datos pasarían a ser **datos personales sensibles** (Art. 3, fracc. VI) y harían falta consentimiento expreso y por escrito (Art. 9), cifrado en tránsito y en reposo, y la retención descrita en la sección 4.

3. Aviso de privacidad

Responsable. Luis Abraham Camacho Durán — abram285@gmail.com

Datos que se recaban. Ninguno del usuario final. La aplicación no pide nombre, correo, ubicación ni credenciales, y no incorpora analítica ni rastreadores de terceros.

Datos que se procesan. Contenido editorial público del blog BabyWolf, y métricas de lectura **simuladas** que no provienen de ninguna persona.

Finalidad. Mostrar noticias del blog de forma sincronizada en teléfono, wearable y televisor, como proyecto académico de la asignatura.

Transferencias. No se transfieren datos a terceros. La comunicación entre dispositivos ocurre en la red local de la laptop y nunca sale de ella.

Derechos ARCO. Cualquier persona puede solicitar el **A**cceso, **R**ectificación, **C**ancelación u **O**posición al tratamiento de sus datos escribiendo a abrajam285@gmail.com. Al no recabarse datos personales de usuarios finales, una solicitud ARCO se responde confirmando que no existe registro alguno asociado a la persona solicitante.

Cambios. Cualquier modificación se publicará en este mismo documento dentro del repositorio.

4. Plan de retención de datos

Dato	Dónde	Cuánto dura	Cómo se elimina
Temas y conteo del blog	Memoria del wearable	Hasta el siguiente refresco (60 s)	Se sobrescriben; nada se persiste
Noticias leídas y minutos	Memoria del wearable	Mientras la app está abierta	Se pierden al cerrarla
Noticia seleccionada	Memoria del hub	Hasta la siguiente selección	Se sobrescribe; el hub no guarda historial
Noticias de la API	Cache del service worker	Hasta el siguiente fetch con red	<code>caches.delete()</code> al cambiar la versión del SW
Estáticos de la PWA	Cache del service worker	Hasta cambiar la versión	El evento <code>activate</code> borra las caches viejas

Nada se escribe a disco. No se usa `localStorage`, ni `IndexedDB`, ni base de datos local en ninguno de los tres dispositivos. Al cerrar las aplicaciones no queda rastro de la sesión, salvo la cache del service worker, que sólo contiene contenido público del blog.

Desinstalar la app o borrar los datos del navegador elimina todo por completo.

5. Checklist de seguridad de la PWA

Control	Estado	Dónde
CSP en meta tag	✓	tv_pwa/index.html
└ default-src 'self'	✓	Todo lo no declarado queda bloqueado
└ script-src 'self'	✓	Sin scripts inline ni CDNs
└ img-src / media-src acotados	✓	Sólo unsplash y el storage del blog
└ connect-src acotado	✓	Sólo la API del blog y el hub local
└ object-src 'none'	✓	Sin plugins embebidos
└ base-uri 'none'	✓	Impide reescribir rutas relativas
HTTPS hacia la API	✓	El backend en Railway sirve sólo TLS
SRI (Subresource Integrity)	N/A — ✓ por diseño	No se carga ningún recurso externo: no hay CDN que verificar. Es una garantía más fuerte que un hash SRI.
Validación de origen	✓	tv_pwa/js/sync.js
Validación de forma del mensaje	✓	validarMensaje()
Sin innerHTML	✓	Todo se pinta con textContent
URLs de imagen restringidas a https	✓	urlSegura() en api.js
Sin secretos en el repositorio	✓	GET /api/posts es público; .gitignore bloquea .env, *.jks, key.properties

Validación de entrada en el hub

El hub también trata como sospechoso todo lo que recibe ([hub/hub.dart](#)): limita el tamaño de los cuerpos a 8 KB, descarta el JSON malformado y exige que cada mensaje traiga los campos esperados con el tipo correcto. Un mensaje que no cumple se ignora en silencio, igual que una trama corrupta en una radio real.

Alumno: Luis Abraham Camacho Durán **Fecha:** 03 de agosto de 2026

Firma: _____